

Report for ADL project
Motorized Automatic Pitcher For Guitar

Bogni Francesco - Sandrini Leonardo

June 30, 2026

Contents

1	Introduction	2
2	Signal reception and processing	3
2.1	Sampling	3
2.2	Downsampling	5
2.3	The ping pong buffer	6
2.4	The FFT IP	6
2.5	Smart peak finder	7
2.6	Outro	9

Chapter 1

Introduction

The integration of digital signal processing (DSP) and automated electromechanical systems offers powerful solutions to real-world challenges. Musicians frequently face the time-consuming task of tuning their instruments, and while digital tuners can provide visual feedback, they still require manual adjustment by the performer. This project introduces an autonomous, hardware-driven solution: an automatic guitar tuning system implemented on a Xilinx Zedboard Development Kit using VHDL.

The audio received directly from the guitar is processed thanks to a Fast Fourier Transform (FFT) algorithm, which identifies the fundamental frequency of the string being played. Such frequency is then compared to the desired target frequency, and allows the system to determine the necessary adjustments to bring the string into tune.

By bypassing general-purpose microprocessors and implementing the core logic directly into the hardware of the Zedboard, the system achieves the low-latency, deterministic performance required for real-time audio analysis and motor control.

Chapter 2

Signal reception and processing

2.1 Sampling

The sampling of the data is performed using the ADAU1761 chip of the Zed-Board. It is a coder/decoder that samples the voltage level coming from the line-in, and can also output it on the line-out and do all sorts of transformations. The process of sampling has been facilitated by the use of the interface developed by Mike Field [2].

In Figure 2.1 and Figure 2.2 you can see the high level overview of the interface. We won't cover it in detail here, but you can check out the github repository [2].

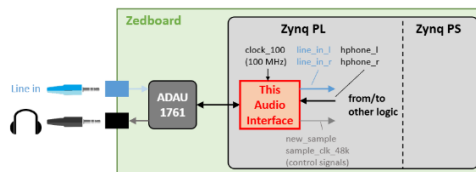


Figure 2.1: Audio interface overview

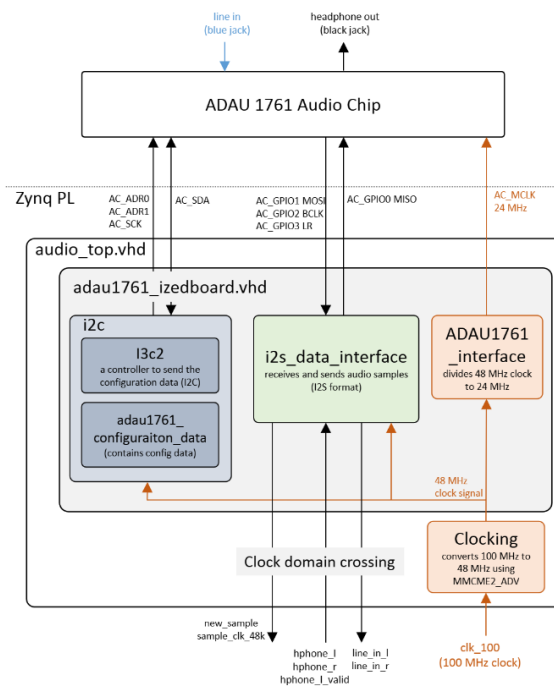


Figure 2.2: Audio interface block diagram.

2.2 Downsampling

We aim to perform the Fast Fourier Transform(FFT) on our samples to detect what is the frequency with the maximum power, i.e. the fundamental frequency. But what resolution can we achieve? What is the range of frequencies we can measure?

When we fill the FFT buffer, we are capturing a specific “window” of time. If we collect N samples, and take a new sample f_s times per second, the total time it takes to fill that buffer is:

$$T = \frac{N}{f_s} \quad (2.1)$$

The FFT assumes that the chunk of data we captured is one repeating cycle. Therefore, the absolute lowest frequency the FFT can see (other than 0 Hz DC) is a wave that completes exactly one full cycle inside our time window T . Thus:

$$f_{\min} = \frac{1}{T} \quad (2.2)$$

Substituting Equation 2.1 into Equation 2.2, we get:

$$f_{\min} = \frac{f_s}{N} = \Delta f \quad (2.3)$$

Since every bin of the FFT is spaced between one another in multiples of Δf .

We derived the equation that tells us the resolution **and** the minimum frequency detectable, but what about the maximum? The Nyquist theorem tells us that the maximum frequency we can detect through sampling is:

$$f_s > 2 \cdot f_{\max} \quad (2.4)$$

So with N fixed, to get a finer resolution (smaller Δf), we want to **lower** f_s , but not too much; otherwise, we will not be able to detect some needed frequencies according to Equation 2.4. (*The role of N will be discussed in Section 2.4*)

Here is where **downsampling** comes into play: We can decide how many samples to keep to achieve a certain resolution (instead of changing the sample rate, which would be inconvenient). But instead of discarding 9 samples out of 10, we accumulate those 10 values into a register and use that as our downsampled value.

This method in addition helps prevent spikes given by physical events (like picking a string) by effectively computing an *unscaled average*. (Since we are looking for the biggest value in the FFT, we do not need to scale it back down).

Moreover, it helps with another problem, **aliasing**: the Nyquist theorem (Equation 2.4) also tells us that if the guitar produces some high frequency noises (which it does), the low f_s will produce some low-frequency noise into the processed data. If you just dropped 9 out of 10 samples, all the frequencies multiple of f_s would alias to 0 Hz (DC), giving power to the incorrect bin. A visual explanation can be found at dsprelated.com[1]. The accumulation helps because it acts as a **low pass filter**:

In signal processing, summing N subsequent samples is the same as **convolving** your audio with a “rectangular window” [3], which in the frequency domain has a frequency response given by the *sinc* function:

$$H(f) = \frac{\sin(\pi f M T_s)}{\sin(\pi f T_s)} \quad (2.5)$$

Where M is the downsample factor (10), and T_s is the sample period.

This solves completely the “alias to 0” problem, because the Equation 2.5 has zeroes at multiples of f_s . Moreover, the low-frequencies pass through with maximum gain ($M = 10$), while the higher frequencies are attenuated. For example at our Nyquist limit:

$$H(2400) = 6.39 \quad (2.6)$$

Which is $\approx 63\%$ of maximum low-frequency amplitude.

2.3 The ping pong buffer

To prevent data loss while the FFT component is elaborating the data, we set up two buffers of 8192 entries in BRAM, and when one is full we start the FFT on that data and start pushing new samples on the other buffer.

2.4 The FFT IP

For performing the FFT we use the Xilinx component set up like that: The output of the FFT are the frequencies components values spread across

8192 bins.

2.5 Smart peak finder

To find the fundamental frequency we want to find the most powerful component. First problem is that the resulting 8192 bins, having the sample rate at 4800 hz, are:

- Bins 0 to 4095 represent the true positive frequencies from 0 Hz to 2,400 Hz (Nyquist Limit).
- Bins 4096 to 8191 represent the negative frequencies wrapping backwards from 4,800 Hz down to 2,400 Hz.

Because the spectrum is mirrored, the peak energy is split equally in half. If one plays a 440 Hz tone, the FFT produces a massive spike at Bin 751 (+440 Hz), but it produces an identical massive spike at Bin 7441 (which is 4,800 Hz - 440 Hz = 4360 Hz) That's why we only search for the bins up to 4096.

But here comes another problem: when playing the notes on a guitar, you don't only get the fundamental frequency, you also get multiples of it, the **harmonics**, which sometimes can have an higher power than the fundamental, breaking the calculation. To get around this we implemented an algorithm called **Harmonic Product Spectrum (HPS)**:

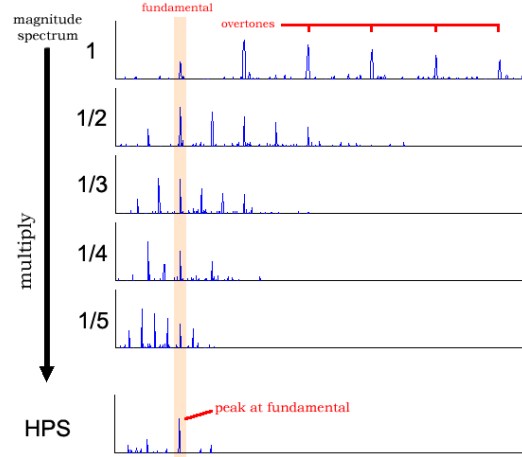
We hold 3 BRAMs, that will contain all the *real* samples of f_s , $2f_s$, $3f_s$ respectively of sizes 4096, 2048, 1366. When the data comes out of the FFT block, it puts all the bins in the first BRAM, one in two bins in the second and one in three for the third BRAM. As a result, the harmonics will line up at the same index, so we can multiply them together and finally

check for the **highest power**. A visual example of this can be seen in Figure 2.3

There are different problems with this approach:

1. The full power calculation of the value of the bin takes 64 bits, and multiplying three of them together would result in a 128 bits value,

Figure 2.3: Example of HPS result. *Credits: sv.mazurka.org.uk*



taking way too many DSP slices over the ZedBoard and risking timing violations.

2. The operations of calculating the power and the HPS cannot be done in the same clock cycle: multiplication takes a long time.

For the **first** problem we first calculate the power with the standard formula, with the difference of avoiding the square root, since we only care about the maximum, there's no need to square it back down:

$$P = Re^2 + Im^2 \quad (2.7)$$

And from this 64 bits value we extract 16 bits, those between 48th and 32nd, which are the ones that carry the most useful information for us. If the full power value is bigger than 2⁴⁸ we set the 16 bits to 1s. If instead the full power value is smaller than 2³² then we set the 16-bit value to 1 to avoid breaking the HPS multiplication.

For the **second** problem we set up a “conveyor belt”: a 4 stage pipeline for the **write**:

1. Split real part and imaginary part.
2. Square each.

3. Add them together.
4. Extract the 16 bit value.

and a 3-stage pipeline for the **HPS**:

1. Multiply value in BRAM1 with value in BRAM2.
2. Multiply last result with value in BRAM3.
3. Compare and save the peak.

2.6 Outro

This is the conclusion to the part of the signal processing, that went from the very first sampling part to finding the fundamental frequency. It did so by downsampling the data to achieve a better resolution, setting up correct buffering to never do useless stalling, performing the FFT, and finding out the fundamental frequency overcoming many different problems. Now that we found that, we send the data to the OLED and the Stepper motor.

Bibliography

- [1] dsprelated.com. <https://www.dsprelated.com/foundations/sampling.php>.
- [2] Mike Field. zedboard audio interface. https://github.com/ems-kl/zedboard_audio, 2015.
- [3] Richard G. Lyons. *Understanding Digital Signal Processing*, section 11.5.